# Label-Constrained Column Annotation with Language Models and Graph Neural Networks

Duo Yang
*KU Leuven, Belgium*
*Flanders Make@KU Leuven, Belgium*
*Leuven.AI, Belgium*
duo.yang@kuleuven.be

Ioannis Dasoulas
*KU Leuven, Belgium*
*Flanders Make@KU Leuven, Belgium*
*Leuven.AI, Belgium*
ioannis.dasoulas@kuleuven.be

Anastasia Dimou
*KU Leuven, Belgium*
*Flanders Make@KU Leuven, Belgium*
*Leuven.AI, Belgium*
anastasia.dimou@kuleuven.be

***Abstract*—Assigning semantic labels to table columns and identifying relations between columns pose significant challenges in data management. Automatic column annotation has been widely treated as classification, with recent works using language models trained on annotated tables with type and property labels. While these language models have effectively modeled individual tables, they often overlook the underlying graph structure of the label space, where constraints can exist between certain types and properties within and across tables. To fill this gap, we propose RODEO, a two-tower architecture that integrates a language model and a graph neural network (GNN) to model the table and semantic labels, respectively. We reformulate column annotation tasks from classification to matching problems, where column and column-pair embeddings are aligned with embeddings that represent their corresponding semantic types (nodes) and properties (edges) within the graph. These embeddings, derived from the language model and GNN, are co-trained end-to-end using triplet loss with an online negative mining strategy. The training process brings semantically related columns and labels closer in the embedding space by minimizing their distances. Our approach, evaluated on publicly available benchmark datasets, outperforms state-of-the-art methods in both column type and column property annotation, highlighting that modeling label constraints through the graph significantly improves overall performance. Ablation studies on the triplet loss and GNN show the robustness of our framework's training procedure.**

## I. Introduction

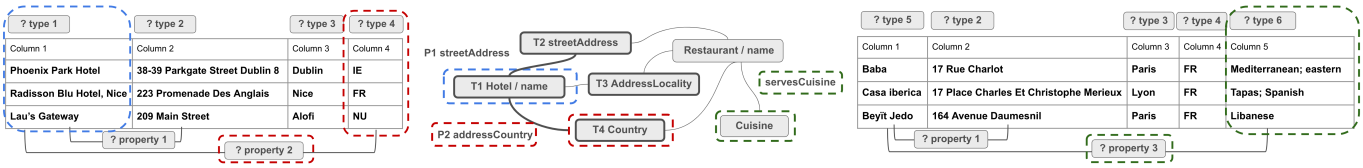
Understanding the semantics of table elements is a crucial first step for data management tasks such as cleaning, integration, discovery, and search. This process is often achieved through table annotation [1, 2, 3], which involves fundamental tasks to analyze tabular data by annotating individual table components, identifying relationships within and between tables, and interpreting tables as complete data units.

Among table annotation tasks, understanding the semantics of table columns is particularly critical, as it enables accurate interpretation and utilization of tabular data. In this work, we focus on **Column Type Annotation (CTA)**, i.e. annotating table columns with semantic types of the values/entities they contain, and **Column Property Annotation (CPA)**, i.e. annotating the relations between two columns with semantic properties, also known as relation extraction. For example, in Fig.1a, column 1 was annotated as "*Hotel/name*" (CTA), and relation between column 1 & 4 was annotated as "*address-Country*" (CPA). Both tasks are typically framed as machine learning classification [4, 5] with a fixed set of pre-defined types and properties as labels.

Previous studies have leveraged advances in neural networks to annotate columns by integrating features from tables [6, 7]. With the rise of pre-training paradigm in recent years, language models [8, 9, 10] were also adapted for table understanding by pre-training [5, 11], and were widely evaluated among various table annotation tasks [12, 13, 14, 15, 4, 16]. These studies have focused mainly on improving the table representation, either by adding more tabular features [6, 7, 17, 18, 19] or perfecting the architecture [12, 13, 14, 15, 4, 16], but the correlation between specific types and properties are not taken into account explicitly for annnotating columns.

To be more specific, a dataset contains restaurant tables with subject columns annotated as "*Restaurant/name*" like Fig.1c. These tables often include context columns annotated with property "*servesCuisine*" and type "*Cuisine*". We can identify a consistent label set from context columns frequently appearing alongside "*Restaurant/name*". Therefore, type "*Restaurant/name*" shows strong correlations with a subset of related properties (types) from the overall label set. The same applies to type "*Cuisine*", only linked to the property "*servesCuisine*" among all properties. With property "*servesCuisine*" and type "*Cuisine*", tables with subject columns as "*Restaurant/name*" can also be distinguished from those with "*Hotel/name*", like between Fig.1a and 1c, since only restaurant tables include such type and property. **A graph is a perfect fit to represent the pair-wise correlations among these labels, where types correspond to nodes and properties to edges, as Fig.1b.** The graph can provide structural constraints [20] to learn such differences within and across tables during training.

In this paper, we introduce a novel embedding-based training framework optimized with triplet loss, **RODEO** (outlined in Fig. 2), that considers both table and graph for column annotation. To achieve embedding alignment between tables and the graph, CTA and CPA are reformulated from classification to matching, focusing on aligning nodes and edges in the graph with columns and column pairs in tables, respectively. In RODEO, the language model generates embeddings for columns and column pairs, while the graph neural network (GNN) models the label space via graph and produces embeddings for nodes and edges.




(a) the "*Hotel*" example table (b) the subgraph view (label space) (c) the "*Restaurant*" example table

Fig. 1: The example SOTAB tables and derived subgraph: column types as nodes, properties (relations) as edges

RODEO is motivated by the goal of advancing data management to address the long-standing challenges of column annotation in real-world scenarios. One important motivation comes from the fact that **the label graph can naturally provide effective inter-table context information that individual tables alone cannot offer during training.**

This can first help in two similar cases: (1) a hard cutoff on wide tables, caused by language model input length limits [21, 22, 23], leads to missing context columns; and (2) individual tables alone may lack context information (columns) by nature [7, 24], making training challenging. For example, the "*Hotel*" and "*Restaurant*" table (See Fig. 1a and 1c) share similar context columns, except the column "*Cuisine*". When this column is cut off or missing in the "*Restaurant*" table, it poses a challenge, as column "*Restaurant/name*" may be misclassified as "*Hotel/name*". In RODEO, when incomplete or context-poor tables are fed into the language model on the table side, their inter-table context (e.g., the label embedding that represents "*Cuisine*") can contribute to the prediction toward target columns (e.g., the column "*Restaurant/name*") and be closely optimized on the graph side together, even if the corresponding columns (e.g., the column "*Cuisine*") are left out in such tables, helping alleviate the problem.

Another case is to distinguish numerical columns across tables in different topics: (3) language models typically struggle to interpret numerical values [25, 26], as such columns often appear similar even when they belong to entirely different topics. Numerical columns are often associated with subject columns, e.g., the property "*checkInTime*" in tables with "*Hotel/name*" versus "*openingHours*" in tables with "*Restaurant/name*". By leveraging structural constraints in labels, model can better differentiate the semantics of numerical columns across diverse topics. For example, the edge "*checkInTime*" may not be reachable within one-hop distance from the node "*Restaurant/name*", but it is directly associated with the node "*Hotel/name*" in the one-hop subgraph.

Also, these two column annotation tasks are optimized independently in previous works [4, 14], although sharing model weights within a multi-task learning framework. This setup still results in (4) two separate label spaces without explicit interactions. For example, the type "Cuisine" and property "servesCuisine" are never jointly optimized in a single training iteration; however, these two labels are naturally correlated. The graph provides a way to integrate the two label spaces. The model can therefore benefit from structural relations captured by the graph during mutli-task learning.

To make such a table-to-graph framework technically feasible, we study an online negative mining strategy that works together with triplet loss. The idea of linking tables to graph has been explored in tasks such as entity linking [27, 28, 29] and graph contrastive learning [30]. However, it has not been thoroughly studied in the context of column annotation, particularly for jointly training language models and graph neural networks in an end-to-end manner. Finding a robust and efficient way to achieve alignment between table and graph for column annotation remains a key challenge, which this work aims to address. Another important aspect is to leverage the powerful capabilities of large language models (LLMs) and examine the framework scalability. While previous studies have primarily relied on instruction-based approaches [31, 32, 33, 34, 35, 36, 37], it is important to investigate the effectiveness of LLM embeddings for column annotation, an area that remains largely unexplored. Therefore, we also scale up language models from BERT [8] to Qwen3 Embedding series [38] to encode the table columns. In summary, the main contributions of this paper are:

1) We propose RODEO, a two-tower architecture of a language model and a graph neural network, which outperforms existing state-of-the-art approaches across benchmarks, SOTAB, WikiTables & GitTables [39, 14, 40];
2) We show that triplet loss, combined with our proposed negative mining strategy, is self-standing and effective for learning representations in semantic column annotation via in-depth comparisons with cross-entropy loss;
3) We show that RODEO, as an embedding-based training framework, is agnostic to the table encoder, supports scaling from *BERT-like* models to large language models.
4) The source code of RODEO, along with the constructed graph triples used in our experiments, is publicly available at https://github.com/dtai-kg/RODEO

Our experiment results show RODEO can handle real-world tabular challenges, such as annotating numerical columns and learning under data scarcity, outperforming the state-of-the-art methods, while adding the graph constraints consistently leads to better performance.

## II. Task Definitions

In this section, we reformulate the CTA and CPA from classification to graph-based matching. We first introduce the graph $G$ as the label space where each $\text{node}_k$ represents a semantic type $t \in V_{\text{type}}$ and each $\text{edge}_m$ represents a

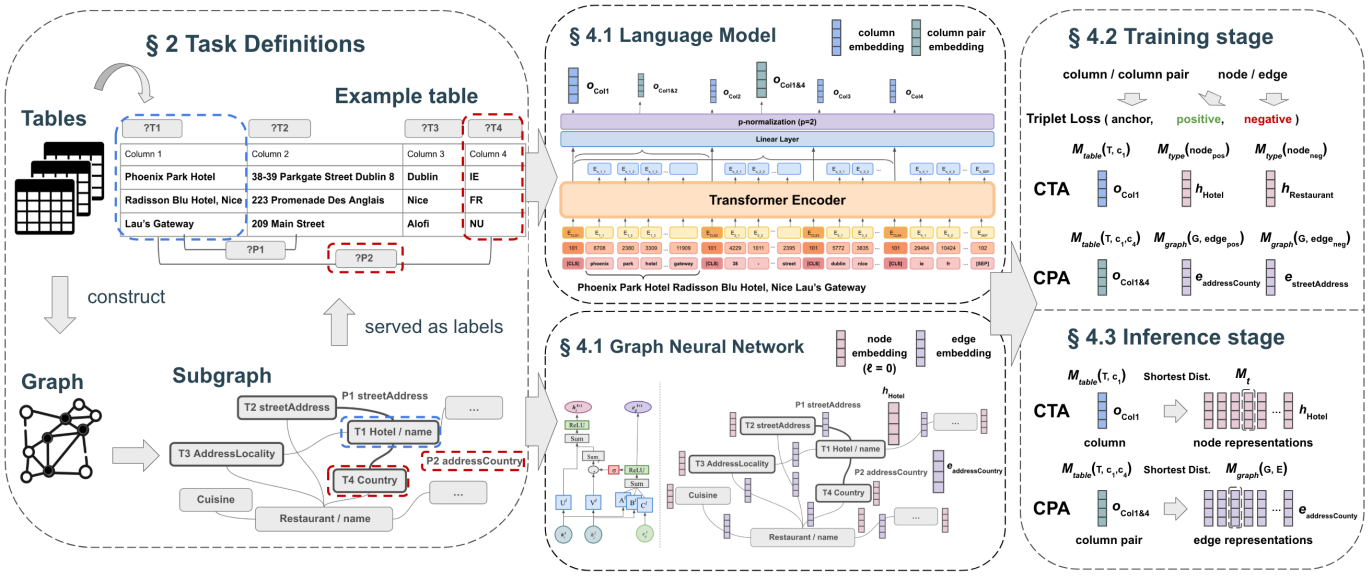



Fig. 2: Overview of RODEO's architecture. (1) The graph captures inter-table information, where semantic labels across different tables are represented as nodes and edges within the same graph, and intra-table information, where semantic labels within a single table frequently appear together as a subgraph. (2) The language model generates embeddings for columns and column pairs, while the GNN learns label representations by taking node embeddings as input and producing edge embeddings as output. (3) During training, the column or column pair embedding serves as the anchor, its corresponding label embedding as the positive, and a negative is selected from the rest; during inference, a type or property is determined by identifying the label embedding closest to the column or column pair embedding.

relationship $r \in V_{\text{property}}$. Unlike the knowledge graph containing entities [27, 29, 41], the graph is constructed at the schema level for both tasks, containing only semantic types and properties. In other words, the graph elements, such as nodes and edges, correspond directly to the label space.

### A. Column Type Annotation

The task is to assign a semantic label to each column in a table, indicating the type of data it contains. In previous studies, it was often defined as follows:

**Definition 1.** *Given a table $T$ with $n$ columns $(c_i)_{i=1}^n$ and a specific column $c_i$ in $T$, the task is to find a column type annotation model $\mathcal{M}_{table}(T, c_i)$ to predict a column type $t$ from a pre-defined vocabulary $V_{type}$ that best describes the semantics of the column $c_i$.*

Accordingly, we reformulate it with the graph as follows:

**Definition 2.** *Given a table $T$ with $n$ columns $(c_i)_{i=1}^n$ and a specific column $c_i$ in $T$, the task is to minimize the distance between the column representation $\mathcal{M}_{table}(T, c_i)$ and node representation $\mathcal{M}_{type}(node_k)$, where the semantic type $t$ that best describes the semantics of the column $c_i$ is represented by the $node_k$ in graph $G$.*

### B. Column Property Annotation

In the context of two columns, the task identifies the properties with column pairs by determining how two columns jointly represent a relationship, often defined as follows:

**Definition 3.** *Given a table $T$ with $n$ columns $(c_i)_{i=1}^n$ and a column pair $(c_i, c_j)$ in $T$, $i \neq j$, the task is to find a column property annotation model $\mathcal{M}_{table}(T, c_i, c_j)$ to predict a relation $r$ from a pre-defined vocabulary $V_{property}$ that best describes the semantics of relation between column $c_i$ and $c_j$.*

Further, we reformulate it with the graph as follows:

**Definition 4.** *Given a table $T$ with $n$ columns $(c_i)_{i=1}^n$ and a column pair $(c_i, c_j)$ in $T$, $i \neq j$, the task is to minimize the distance between the column pair $\mathcal{M}_{table}(T, c_i, c_j)$ and edge representation $\mathcal{M}_{graph}(G, edge_m)$, where the semantic property $r$ that best describes the semantics of relation between the column $c_i$ and $c_j$ is represented by the $edge_m$ in graph $G$.*

The same property can appear across different type pairs, resulting in multiple edges. For example, in Fig.1a and 1c, the property "addressCountry," which connects the "Hotel" and "Country" columns, also applies to the relationship between the "Restaurant" and "Country" columns. As the model first identifies the edge and then determines the corresponding property, task definition 4 remains accurate.

## III. Preliminaries

In this section, we provide a concise overview of neural backbones for table and semantic label representations (section III-A & III-B), along with triplet loss (section III-C) for jointly training these backbones. These introduced formulas will be further reused in Section IV.

TABLE I: Formal Notations

| Symbol | Domain | Description |
|---|---|---|
| $T$ | $-$ | All the columns in a table |
| $c$ | $c_i \in T$ | Column |
| $t$ | $t_k \in V_{\text{type}}$ | Semantic type |
| $r$ | $r_m \in V_{\text{property}}$ | Semantic property (relation) |
| $\lvert V_{\text{type}}\rvert$ | $\lvert V_{\text{type}}\rvert \in \mathbb{N}$ | Vocabulary size of semantic types |
| $\lvert V_{\text{property}}\rvert$ | $\lvert V_{\text{property}}\rvert \in \mathbb{N}$ | Vocabulary size of semantic properties |
| $M_t$ | $M_t \in \mathbb{R}^{\lvert V_{\text{type}}\rvert \times d}$ | Learnable input embedding matrix for semantic types |
| $M_r$ | $M_r \in \mathbb{R}^{\lvert V_{\text{property}}\rvert \times d}$ | Learnable input embedding matrix for semantic properties |
| $G$ | $G = (V, E)$ | Graph (undirected) representing relations between types and properties |
| $node$ | $node_k \in V$ | Node in the graph, representing a semantic type |
| $edge$ | $edge_m \in E$ | Edge in the graph, representing a semantic property (relation) |
| $\lvert V\rvert$ | $\lvert V\rvert \in \mathbb{N}$ | Number of nodes in the graph $G$ |
| $\lvert E\rvert$ | $\lvert E\rvert \in \mathbb{N}$ | Number of edges in the graph $G$ |
| $\sigma$ | $\sigma \in [0, 1]$ | A ratio defining different negative strategies across batches within an epoch |
| | **Equation(s)** | **Description** |
| $\mathcal{M}_{\text{table}}$ | (2), (3), (11) & (12) | Neural backbone to map columns and column pairs in table $T$ to embeddings |
| $\mathcal{M}_{\text{graph}}$ | (4), (5), (6), (13) & (14) | Neural backbone to generate node and edge embeddings based on graph $G$ |
| $\mathcal{M}_{\text{type}}$ | (13) | Lookup function that retrieves type embeddings from learnable matrix $M_t$ |
| $\mathcal{M}_{\text{property}}$ | (14) | Lookup function that retrieves property embeddings from learnable matrix $M_r$ |
| $\mathcal{M}_{\text{label}(\cdot)}$ | $-$ | Simplifed expression for $\mathcal{M}_{\text{type}}(\cdot)$ and $\mathcal{M}_{\text{graph}}(G, \cdot)$ |

### A. Pre-trained Language Model

Pre-trained language models were adapted for table understanding using row- or column-based serialization [4, 14, 5]. DODUO employed the column-based method, treating each table as a sequence of columns with aggregated values [4]. Within column-based serialization, for each table $T$ that has $n$ columns $(c_i)_{i=1}^{n}$, where each column $c_i$ has $m$ cell values $v$, table content are serialized into the sentence $T'$ as follows:

$$T' ::= [\text{CLS}_1]\ v_{11}\ \cdot\cdot\ [\text{CLS}_n]\ v_{n1}\ \cdot\cdot\ v_{nm}\ [\text{SEP}] \tag{1}$$

The i-th column embedding $o_t$ and pair embedding $o_r$ are defined as follows, with [CLS] representing the entire column:

$$o_t = f_{\text{pooler}_{\text{cta}}}\left(\mathcal{LM}\left(T'\right),\ p_{[\ \text{CLS}_i]}\right) \tag{2}$$

$$o_r = f_{\text{pooler}_{\text{cpa}}}\left(\mathcal{LM}\left(T'\right),\ p_{[\ \text{CLS}_1]},\ p_{[\ \text{CLS}_i]}\right) \tag{3}$$

where $o_t, o_r \in \mathbb{R}^d$, $\mathcal{LM}$ denotes a language model, the pooler function $f$ encodes embeddings based on column positions $p$.

### B. Graph Neural Network

Gated Graph Convolutional Network (GatedGCN) [42, 43] built upon the foundation of graph convolutional networks [44] with gate mechanisms [45], residual connections [46] and batch normalization [47]. It follows: $h_i^{(\ell)}$ is denoted as the feature vector at layer $(\ell)$ associated with node $i$, the updated $h_i^{(\ell+1)}$ at the next layer $(\ell+1)$ is obtained by applying non-linear transformations to the central vector $h_i^{(\ell)}$ and feature vectors $h_j^{(\ell)}$ for all nodes $j$ in the neighborhood of node $i$,

$$h_i^{(\ell+1)} = h_i^{(\ell)} + f_{\text{ReLU}}\left(f_{\text{BN}_h}\left(U^{(\ell)}h_i^{(\ell)} + \sum_{j\in N_i} e_{ij}^{(\ell)} \odot V^{(\ell)}h_j^{(\ell)}\right)\right) \tag{4}$$

where $U^{(\ell)}, V^{(\ell)} \in \mathbb{R}^{d\times d}$, $\odot$ refers the Hadamard product, and the edge gate $e_{ij}^{(\ell)}$ is defined as follows:

$$e_{ij}^{(\ell)} = \frac{f_{\text{sig}}(\hat{e}_{ij}^{(\ell)})}{\sum_{j'\in N_i} f_{\text{sig}}(\hat{e}_{ij'}^{(\ell)}) + \epsilon}, \tag{5}$$

$$\hat{e}_{ij}^{(\ell)} = \hat{e}_{ij}^{(\ell-1)} + f_{\text{ReLU}}(f_{\text{BN}_e}(A^{(\ell)}h_i^{(\ell-1)} + B^{(\ell)}h_j^{(\ell-1)} + C^{(\ell)}\hat{e}_{ij}^{(\ell-1)})) \tag{6}$$

where $f_{\text{sig}}$, $f_{\text{ReLU}}$ represent non-linear activation functions, $\epsilon$ is a small constant value for numerical stability, $A^{(\ell)}, B^{(\ell)}, C^{(\ell)} \in \mathbb{R}^{d\times d}$. The architecture allows us to input the learnable node vector $h_i^{(\ell_0)}$ and edge vector $e_{ij}^{(\ell_0)}$ simultaneously, and it considers the graph's structural dependencies during the training. GatedGCN is a specific variant within the broader category of graph neural networks, for simplicity; we refer to GatedGCN as GNN throughout this paper.

### C. Triplet Loss & Negatives Mining

Triplet loss was first introduced in FaceNet [48] to distinguish human faces based on embeddings. As the tasks are reformulated as matching problems based on embeddings, triplet loss is more suitable than cross-entropy loss. The loss function $\mathcal{L}$ is defined often using the Euclidean distance:

$$\mathcal{L}_i\ =\ \max\left(\ 0,\ \|f(x^a) - f(x^p)\|_2^2\ -\ \|f(x^a) - f(x^n)\|_2^2\ +\ \alpha\ \right) \tag{7}$$

where $x^a$ represents an anchor input, $x^p$ is a positive input considered similar to $x^a$ while $x^n$ is a negative input not similar to $x^a$. The index $i$ represents the individual triplet index. $\alpha$ is the margin between positive and negative pairs, and $f$ is the embedding function, the model minimizes the overall loss within a batch:

$$\mathcal{L} = \sum_{i=1}^{|\text{Batch}|} \mathcal{L}_i(x^a, x^p, x^n) \tag{8}$$

One important aspect of triplet loss training is the identification of difficult negatives, denoted as $x^n$, through a process known as **negative mining**, as negatives can help the model to distinguish between similar yet distinct inputs. The negatives that satisfy the following condition, called **hardest negatives**, are considered optimal as they provide informative signals to the model due to their close proximity to the anchor, which needs to be pushed away. Here, the indices $i$ denote the position of $x^n$ among the negative candidates.

$$\arg\min_x \sum_{i=1}^{n} \|f(x^a) - f(x_i^n)\|_2^2 \tag{9}$$

However, training with these hardest negatives can be challenging, as it can sometimes trap the model in poor local minima, resulting in lower performance. Thus, the concept of **semi-hard negatives** was introduced [48], which suggested choosing $x_i^n$ such that

$$\arg\min_{x} \sum_{i=1}^{n} \|f(x^a) - f(x^p)\|_2^2 - \|f(x^a) - f(x_i^n)\|_2^2 \tag{10}$$

Semi-hard negatives, while farther from the anchor than the positive, are still challenging enough as the squared distance is close to that of anchor-positive pair, falling within the margin.

## IV. Methodology

In this section, we describe how columns, column pairs, nodes, and edges are represented in RODEO (section IV-A), how to co-train their embeddings end-to-end with triplet loss (section IV-B), and how to infer types and properties (section IV-C). The remaining subsections present the model variants.

### A. RODEO's Model Architecture

RODEO includes two neural backbones: one leverages a language model to encode table and generate embeddings for columns and column pairs, the other utilizes a graph neural network (GNN) to learn semantic label representations.

*Table Representation* Given a table, the column-based serialization approach [4], as outlined in the previous section, serves as the foundation. For each column and column pair in table $T$, their embeddings are structured as follows:

$$\mathcal{M}_{\text{table}}(T, c_i) = o_t \tag{11}$$

$$\mathcal{M}_{\text{table}}(T, c_i, c_j) = o_r \tag{12}$$

$\mathcal{M}_{\text{table}}$ refers to the model for encoding table, $\mathcal{M}_{\text{table}}(T, c_i)$ denotes the embedding of column $c_i$ from the language model as detailed in Equation (2), and $\mathcal{M}_{\text{table}}(T, c_i, c_j)$ represents the embedding of column pair $c_i$ and $c_j$ generated by the same language model as shown in Equation (3). The pooler layer adjusts their dimension to match the label representation.

*Semantic Label Representation* Node embeddings and edge embeddings together form semantic label representations, with node embeddings serving as input to the GNN and edge embeddings generated as its output. These embeddings are structured as follows:

$$\begin{cases} * \ \mathcal{M}_{\text{type}}\ (node_k) & = h_k^{(0)}, \quad \text{if } \ell = 0 \\ \ \ \mathcal{M}_{\text{graph}}(G, node_k) & = h_k^{(\ell)}, \quad \text{if } \ell \neq 0 \end{cases} \tag{13}$$

$$\begin{cases} \ \ \mathcal{M}_{\text{property}}(edge_m) & = \hat{e}_m^{(0)}, \quad \text{if } \ell = 0 \\ * \ \mathcal{M}_{\text{graph}}(G, edge_m) & = \hat{e}_m^{(\ell)}, \quad \text{if } \ell \neq 0 \end{cases} \tag{14}$$

Where $\mathcal{M}_{\text{graph}}$ refers to GNN for modeling constraints between types and properties, $h_k^{(\ell)}$ denotes the node embedding at layer $\ell$ computed by GNN, in Equation (4); and $\hat{e}_m^{(\ell)}$ represents the edge embedding at layer $\ell$ computed by the same GNN, in Equation (6). Node input embedding $h_k^{(0)}$ and edge input embedding $\hat{e}_m^{(0)}$, are first initialized using Xavier uniform distribution [49] from scratch, where each type and property is represented by a $d$-dimensional learnable vector within type matrix $M_t \in \mathbb{R}^{|V_{\text{type}}| \times d}$ and property matrix $M_r \in \mathbb{R}^{|V_{\text{property}}| \times d}$. While nodes are unique as types ($|V| = |V_{\text{type}}|$), edges may share the same property in the graph ($|E| \geq |V_{\text{property}}|$). Therefore, edges that share the same property are assigned the same edge input embeddings from property matrix $M_r$.

All the embeddings ( $o_t, o_r, h_k^{(0)}, \hat{e}_m^{(\ell)}$ ) eventually lie on a $d$-dimensional space $\mathbb{R}^d$ and are normalized, e.g., $\|o_t\|_2 = 1$, so it stabilizes training by maintaining consistent magnitudes.

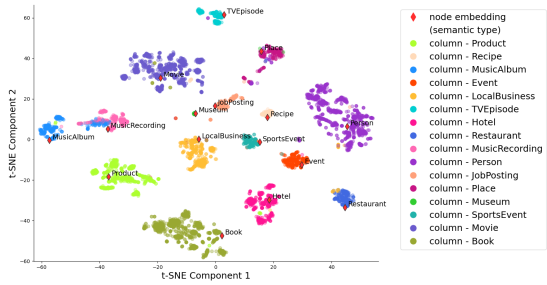



Fig. 3: t-SNE visualization of column & node embeddings.

### B. Training Stage

We demonstrate how these embeddings are enabled within the architecture and contribute to the training within the multi-task learning framework (section IV-B1). Within the scope of column annotation, we define the anchor, positive, and negative for triplet loss (section IV-B2) and propose an online mining strategy on how to select negatives (section IV-B3).

*1) Multi-task Learning Paradigm:* The multi-task learning workflow follows an iterative manner, starting with training one epoch for CTA and then switching to CPA for next epoch (Alg. 1, lines 5-6). The training procedure continues alternating between tasks until the model converges or reaches a specified number of epochs. These two tasks are trained with separate task objectives while sharing the model parameters, not limted to the language model $\mathcal{M}_{\text{table}}$ like [4], but also including the type matrix $M_t \in \mathbb{R}^{|V_{\text{type}}| \times d}$ (all the type label embeddings). In RODEO, we treat CTA as an upstream task and CPA as a downstream task, as a column serves as an independent table unit while column pair relation are only meaningful when individual columns are identified as types. This can be achieved by optimizing their corresponding embeddings from different layers within the GNN. During CTA training, node input embeddings from $M_t \in \mathbb{R}^{|V_{\text{type}}| \times d}$ are chosen for optimization with the targeted column embedding. During CPA training, the edge embeddings $\hat{e}_m^{(\ell)}$ from the final layer of the GNN are chosen along with the targeted column pair embedding. These choices, $\mathcal{M}_{\text{type}}(\cdot)$ and $\mathcal{M}_{\text{graph}}(G, \cdot)$, are marked with a special star ($*$) in Equations (13) & (14), both unified as $\mathcal{M}_{\text{label}}(\cdot)$ in Algorithm 1.

In this design, the type matrix $M_t \in \mathbb{R}^{|V_{\text{type}}| \times d}$ serves as a bridge to better facilitate information sharing between the tasks: (1) in the first training epoch for CTA, the node input embeddings from type matrix $M_t$ retain information solely from the language model; (2) the optimized type matrix $M_t$ provides a solid foundation for CPA because it later serves

**Algorithm 1:** Training Procedure with Online Negative Mining (w/ GNN)

**Input:** Number of epochs $N_{\text{epoch}}$; For task $\in \{\text{cta}, \text{cpa}\}$, training set $D_{\text{task}}$, loss function $\mathcal{L}_{\text{task}}$, optimizer $O_{\text{task}}$, and negative ratio $\sigma_{\text{task}}$

**Output:** Two-tower matching model $\mathcal{M} = (\mathcal{M}_{\text{table}}, \mathcal{M}_{\text{graph}})$, where:

- $\mathcal{M}_{\text{table}} = (\mathcal{LM}, \{f_{\text{pooler}_{\text{cta}}}, f_{\text{pooler}_{\text{cpa}}}\})$ refers to table representation model,
- $\mathcal{M}_{\text{graph}} = (\mathcal{M}_{\text{GatedGCN}}, M_t, M_r)$ refers to graph representation model, including that $\mathcal{M}_{\text{type}} = (M_t)$ node input embeddings
- $\mathcal{M}_{\text{GatedGCN}} = (U, V, A, B, C, \text{BN}_h, \text{BN}_e)$ with parameters $U, V, A, B, C$ and batch normalization terms $\text{BN}_h, \text{BN}_e$.

1 Initialize language model $\mathcal{LM}$ using pre-trained weights;
2 Initialize poolers $\{f_{\text{pooler}_{\text{cta}}}, f_{\text{pooler}_{\text{cpa}}}\}$ randomly;
3 Initialize graph neural network $\mathcal{M}_{\text{GatedGCN}}$ randomly;
4 Initialize graph input matrices $M_t$ and $M_r$ using Xavier uniform distribution;
5 **for** $ep = 1$ **to** $N_{epoch}$ **do**
6  **for** $task \in \{cta, cpa\}$ **do**
7   $0 \leftarrow \text{count}_{\text{hardest(task)}}, \text{count}_{\text{semi-hard(task)}}$ ;
8   Randomly split $D_{\text{task}}$ into mini-batches $B_{\text{task}}$ ;
9   **for** $batch \in B_{task}$ **do**
10    set $x^a \leftarrow$ column $c_i$ **if** $task = cta$ **else** column pair $(c_i, c_j)$ ; // anchor
11    set $x^p \leftarrow$ node for semantic type $t_{\text{label}}$ **if** $task = cta$ **else** edge for semantic property $r_{\text{label}}$ ; // positive
12    set $\{x_i^n \mid x_i^n \neq x^p\} \leftarrow$ nodes for types $\{t_{\text{negative}}\}$ **if** $task = cta$ **else** edges for properties $\{r_{\text{negative}}\}$ ;
13    set size $|S| \leftarrow (|V| - 1)$ **if** $task = cta$ **else** $(|E| - 1)$ ; // negative mining space
14    set $\mathcal{M}_{\text{label}}(\cdot) \leftarrow \mathcal{M}_{\text{type}}(\cdot)$**if** $task = cta$ **else** $\mathcal{M}_{\text{graph}}(G, \cdot)$
15    **if** $\sigma_{task} > count_{hardest(task)}/(count_{hardest(task)} + count_{semi\text{-}hard(task)} + 1)$ **then**
16     set $x^n \leftarrow \arg\min_{x^n} \sum_{i=1}^{|S|} \|\mathcal{M}_{\text{table}}(T, x^a) - \mathcal{M}_{\text{label}}(x_i^n)\|_2^2$
17     $\text{count}_{\text{hardest(task)}}$ ++ ; // select the hardest negative
18    **else**
19     set $x^n \leftarrow \arg\min_{x^n} \sum_{i=1}^{|S|} ( \|\mathcal{M}_{\text{table}}(T, x^a) - \mathcal{M}_{\text{label}}(x^p)\|_2^2 - \|\mathcal{M}_{\text{table}}(T, x^a) - \mathcal{M}_{\text{label}}(x_i^n)\|_2^2 )$
20     $\text{count}_{\text{semi-hard(task)}}$ ++ ; // select semi-hard negative
21    $\mathcal{L}_{\text{task}} \leftarrow \mathcal{L}(\mathcal{M}_{\text{table}}(T,\ x^a),\ \mathcal{M}_{\text{label}}(\ x^p),\ \mathcal{M}_{\text{label}}(\ x^n))$ ; // anchor, positive, negative
22    $\mathcal{M} \leftarrow$ back_propagate$(\mathcal{M}, O_{\text{task}}, \partial\mathcal{L}_{\text{task}}/\partial(\mathcal{LM} \oplus f_{\text{pooler}_{\text{task}}} \oplus \mathcal{M}_{\text{label}}))$;
23 **return** $\mathcal{M}$ ;

as node inputs to the GNN, contributing to the final edge representations; (3) during CPA training, the type matrix $M_t$ gets optimized through back-propagation and learns structural constraints between nodes; (4) the optimized type matrix $M_t$, enriched with more semantic information from the previous CPA training, enhances the CTA training in the next epoch.

*2) Triplet Loss:* Triplet loss is used in RODEO for end-to-end training. Each training example consists of an anchor, a positive, and a negative to form a triplet as the optimized target. In the context of semantic table annotation, the training triplets are structured as follows (Alg. 1, lines 10-12): (1) the targeted column or column pair embedding serves as the anchor; (2) the node or edge embedding that represents the semantics of the targeted column or column pair functions as the positive; (3) the remaining node or edge embeddings form the negative mining space. Only one negative is eventually selected from this space. These triplets (embeddings) are grouped into batches for training, as follows:

$$\mathcal{L}_{cta} = \sum_{i=1}^{|\text{Batch}|} \mathcal{L}_i(\mathcal{M}_{\text{table}}(T,\ x^a),\ \mathcal{M}_{\text{type}}(\ x^p),\ \mathcal{M}_{\text{type}}(\ x^n)) \tag{15}$$

$$\mathcal{L}_{cpa} = \sum_{i=1}^{|\text{Batch}|} \mathcal{L}_i(\mathcal{M}_{\text{table}}(T,\ x^a),\ \mathcal{M}_{\text{graph}}(G,\ x^p),\ \mathcal{M}_{\text{graph}}(G,\ x^n)) \tag{16}$$

Here, $x^a$ refers to the targeted column $c_i$ or column pair $(c_i, c_j)$, $x^p$ refers to the ground truth node for the type or the ground truth edge for the property, $x^n$ refers to selected node or edge from the negative mining space $|S|$. This ensures that the embedding of node or edge representing the semantics of column or column pair (positive) is brought closer to the anchor embedding, while the embeddings of those dissimilar nodes or edges (negatives) are pushed further away.

*3) Online Negative Mining:* Online negative mining refers to the dynamic process of selecting negatives during the training procedure [48] instead of preparing all the training triplets before the first epoch begins. Selecting the informative negatives for completing the training triplets is a crucial step to ensure model training stability and evaluation performance. In the context of column type annotation, given a targeted column, the negative mining space $|S|$ includes all the nodes except the one that represents the semantics of the targeted column (Alg. 1, lines 12-13). Only one negative node is selected from the mining space based on the shortest distance, following either the semi-hard strategy in Equation (17) or the hardest negative strategy in Equation (18).

$$\arg\min_{x^n} \sum_{i=1}^{|S|} ( \|\mathcal{M}_{\text{table}}(T, x^a) - \mathcal{M}_{\text{type}}(x^p)\|_2^2 - \|\mathcal{M}_{\text{table}}(T, x^a) - \mathcal{M}_{\text{type}}(x_i^n)\|_2^2 ) \tag{17}$$

$$\arg\min_{x^n} \sum_{i=1}^{|S|} \|\mathcal{M}_{\text{table}}(T, x^a) - \mathcal{M}_{\text{type}}(x_i^n)\|_2^2 \tag{18}$$

Along with the selected negative $x^n$ from a specific strategy (Alg. 1, lines 16 & 19), the training triplet is completed and serves as the optimized target (Alg. 1, line 21).

$$\arg\min_{x^n} \sum_{i=1}^{|S|} ( \|\mathcal{M}_{\text{table}}(T, x^a) - \mathcal{M}_{\text{graph}}(G, x^p)\|_2^2 - \|\mathcal{M}_{\text{table}}(T, x^a) - \mathcal{M}_{\text{graph}}(G, x_i^n)\|_2^2 ) \tag{19}$$

$$\arg\min_{x^n} \sum_{i=1}^{|S|} \|\mathcal{M}_{\text{table}}(T, x^a) - \mathcal{M}_{\text{graph}}(G, x_i^n)\|_2^2 \tag{20}$$

Regarding CPA, the anchor $x^a$ is the targeted column pair,

and the mining process works with edges and their embeddings from the GNN instead of nodes and their embeddings by replacing $\mathcal{M}_{\text{type}}(\cdot)$ with $\mathcal{M}_{\text{graph}}(G, \cdot)$. In addition, we unify the above two functions as $\mathcal{M}_{\text{label}}(\cdot)$ just to simplify the expressions (Alg. 1, line 14).

In this paper, we unify the negative mining strategies by balancing the semi-hard and the hardest negatives in varying proportions (Alg. 1, lines 15 - 20). Batches are optimized by alternating between the semi-hard and the hardest negatives, guided by a pre-defined ratio $\sigma$, which we introduced as a training hyperparameter. When the ratio $\sigma$ is zero, the training exclusively uses semi-hard negatives for all the epochs. Conversely, when it is one, the training relies entirely on the hardest negatives. The ratio can vary within the range [0, 1]. In either case, both the semi-hard and the hardest negatives are incorporated into the training process, ensuring that batches are optimized iteratively between them with the defined ratio $\sigma$. The negative ratios can also be adjusted differently in multi-task learning, as the model's sensitivities to the semi-hard and hardest negatives may differ between tasks.

### C. Inference Stage

During the inference, the well-trained neural backbones are employed to generate embeddings for columns, column pairs, nodes, and edges. The semantic type or property is then determined by identifying the embedding with the shortest distance to the target representation from the language model.

*Column Type Annotation* The goal is to determine the semantic type by finding the node in the set $V$ whose embedding is closest to the embedding of column $c_i$, such that

$$\arg\min_{\text{node}} \sum_{k=1}^{|V|} \|\mathcal{M}_{\text{table}}(T, c_i) - \mathcal{M}_{\text{type}}(node_k)\|_2^2 \tag{21}$$

Once the matching node is found, its corresponding type $t$ is retrieved from $V_{\text{type}}$ and annotated to the targeted column $c_i$.

*Column Property Annotation* The goal is to identify the semantic property by finding the edge in the set $E$ whose embedding is closest to the embedding of the relationship between columns $c_i$ and $c_j$, as defined by

$$\arg\min_{\text{edge}} \sum_{m=1}^{|E|} \|\mathcal{M}_{\text{table}}(T, c_i, c_j) - \mathcal{M}_{\text{graph}}(G, edge_m)\|_2^2 \tag{22}$$

Once the edge that satisfies this condition is identified, its corresponding semantic property $r$ is retrieved from $V_{\text{property}}$ and annotated to the relationship between columns $c_i$ and $c_j$.

### D. Model Architecture w/o GNN

Given the complexity of RODEO, which integrates triplet loss training and a graph neural network, it is necessary to decouple the components for step-by-step ablation studies. Therefore, we define a new variant that excludes the GNN. It enables a two-layer analysis, detailed in Section VI, which includes (1) a direct performance comparison between triplet loss and cross-entropy loss, as employed by DODUO [4], and (2) an evaluation of model variants with and without the GNN, where the latter considers label constraints through the graph.

In this variant, the model $\mathcal{M}_{\text{graph}}$ for semantic label representation is omitted. The primary change lies in CPA, where the property matrix $M_r \in \mathbb{R}^{|V_{\text{property}}| \times d}$ is used directly for matching instead of the final edge representation. During inference, this variant omits the graph $G$ as follows:

$$\arg\min_{t} \sum_{k=1}^{|V_{\text{type}}|} \|\mathcal{M}_{\text{table}}(T, c_i) - \mathcal{M}_{\text{type}}(t_k)\|_2^2 \tag{23}$$

$$\arg\min_{r} \sum_{m=1}^{|V_{\text{property}}|} \|\mathcal{M}_{\text{table}}(T, c_i, c_j) - \mathcal{M}_{\text{property}}(r_m)\|_2^2 \tag{24}$$

### E. Model Architecture w/ metadata & topics

Given that not all the collections have property annotations available for column pairs and not all column pairs exhibit explicit relationships in a real-world setting, we construct the graph on synthesized topics and show how it works in Fig. 4.

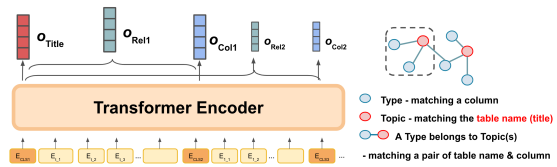



Fig. 4: Architecture w/ metadata & synthesized topics

In this variant, tables are clustered based on how type labels appear together. Specifically, each table is represented as a binary vector showing which type labels are present. Each table belongs to one cluster (synthesized topic). The synthesized topics are also added as nodes in the graph. Connections are made between topics and types in the graph, since each type naturally belongs to one or more topics. The language model also uses table metadata, generating a single embedding for the table name. This embedding is matched to one of the topic nodes, similar to how columns are linked to type nodes. In this setting, relationship prediction is introduced as an auxiliary task to improve type annotation.

### F. Model Architecture w/ LLM Embeddings

To showcase that RODEO, as an embedding-based training framework that is agnostic to the table encoder, supports scaling from language models like BERT to large language models that are capable of generating embeddings, we leverage Qwen3 Embeddings [38] to encode table columns. This enables a fair comparison with other LLM-based approaches in terms of model scale and examines the scalability of our framework.

In this variant, only the model $\mathcal{M}_{\text{table}}$ for table representation is upgraded. Similar to Equation 1, table content are serialized into the sentence $T'$ but with <|endoftext|> as EOS token inserted after each column:

$$T' ::= [\text{BOS}]\ v_{11}\ \cdot\cdot\ [\text{EOS}_1]\ v_{n1}\ \cdot\cdot\ v_{nm}\ [\text{EOS}_n] \tag{25}$$

Similar to Equations 2 and 3, we apply linear transformations to the embeddings, followed by RMSNorm [50] to stabilize the model training. All other neural components and the training procedure remain unchanged within the RODEO framework.

## V. Evaluation

In this section, we describe the datasets and graph construction, then cover evaluations including the experimental setup, comparisons with baselines, and key findings from the results.

### A. Datasets and Graph Construction

We use three large-scale benchmark datasets, SOTAB [39], WikiTables [14], and GitTables [40] for CTA and CPA tasks. To enable model training, schema graphs are constructed beforehand from the annotations ($\mathcal{A}_{cta}$, $\mathcal{A}_{cpa}$; test set excluded) through an automated process driven by dataset statistics, supplemented by minimal human verification. In datasets, annotation files are usually provided separately from the tables where CTA entries are tuples $(T, i, l_t)$ assigning type $l_t$ to column $i$ of table $T$, and CPA entries are tuples $(T, i_{subj}, i_{obj}, l_r)$ assigning property $l_r$ between subject and object columns.

**Algorithm 2:** Schema Graph Construction

**Input:** Column annotation files $\mathcal{A}_{cta}$ with tuples $(T, i, l_t)$ and $\mathcal{A}_{cpa}$ with tuples $(T, i_{subj}, i_{obj}, l_r)$; thresholds $\tau_{\text{sp}}, \tau_{\text{po}}$
**Output:** Schema graph $G$

1 Initialize triple multiset $\mathcal{M}$; build lookup $\mathcal{F} : (T, i) \rightarrow l_t$ from $\mathcal{A}_{cta}$;
2 **foreach** $(T, i_{subj}, i_{obj}, l_r) \in \mathcal{A}_{cpa}$ **do**
3  **if** $\mathcal{F}(T, i_{subj})$ *and* $\mathcal{F}(T, i_{obj})$ *exist* **then**
4  Add $(\mathcal{F}(T, i_{subj}),\ l_r,\ \mathcal{F}(T, i_{obj}))$ as $(s, p, o)$ to $\mathcal{M}$ ; `// main`
5  **else if** $\mathcal{F}(T, i_{subj})$ *exists* **then**
6  Add $(\mathcal{F}(T, i_{subj}),\ l_r,\ l_{synthetic_type})$ as $(s, p, o)$ to $\mathcal{M}$;
7 **foreach** $(T, i_{subj}) \in \mathcal{A}_{cpa}$ **do**
8  Let $\mathcal{O}_T \leftarrow \{\mathcal{F}(T, i) \mid (T, i, l_t) \in \mathcal{A}_{cta}\}$;
9  **foreach** $o_t \in \mathcal{O}_T$ **do**
10  **if** $\mathcal{F}(T, i_{subj}) \neq o_t$ *and* $(\mathcal{F}(T, i_{subj}), o_t) \notin \mathcal{M}$ **then**
11  Add $(\mathcal{F}(T, i_{subj}),\ l_{synthetic_property},\ o_t)$ as $(s, p, o)$ to $\mathcal{M}$;
12 Count co-occurrences $c_{\text{sp}}$, $c_{\text{po}}$ over $(s, p)$ and $(p, o)$ in $\mathcal{M}$;
13 $G \leftarrow \{(s, p, o) \mid c_{\text{sp}}(s, p) \geq \tau_{\text{sp}} \wedge c_{\text{po}}(p, o) \geq \tau_{\text{po}}\}$;
14 Restore valid triples via manual verification;
15 **return** $G$;

The construction of schema graphs starts with the semantic types of subject columns $\mathcal{F}(T, i_{subj})$, which serve as entry points connected to the types of other object columns $\mathcal{F}(T, i_{obj})$ through property relations $l_r$ in the tables (Alg. 2, lines 3-4). Since the same types can appear in object columns across different tables, the resulting connections extend beyond individual tables, capturing the inter-table context. To address sparsity and maintain proper connectivity in the graph, we also introduce synthetic types and properties, not explicitly presented in the vocabulary. Specifically, we introduce synthetic types (Alg. 2, lines 5-6) when object columns lack type labels, and add new properties (Alg. 2, lines 7-11) to capture implicit relationships between co-existing types within the same table. We analyze the co-occurrence matrices ($c_{\text{sp}}$, $c_{\text{po}}$) between type and property annotations associated with these columns. To filter out low-confidence triples, we set a threshold $\tau$ for each matrix and manually verify data in case valid triples are wrongly removed. These final triples $(s, p, o)$ form a schema graph $G$. Similar to the synthesized approach in SANTOS [51], our construction leverages co-occurrence among existing annotations, rather than using ontologies directly, as the annotations do not strictly align with ontology restrictions like domains and ranges [52, 51].

Table II provides an overview of the dataset statistics:

TABLE II: Dataset statistics

| Dataset | annotation | | vocabulary | | graph (ours) | | |
|---|---|---|---|---|---|---|---|
| | columns | column pairs | col types | col rels | nodes | edges | triples |
| SOTAB* | 120,507 | 114,793 | 82<br>46 | 108<br>49 | 105<br>59 | 129<br>64 | 356<br>228 |
| WikiTables | 654,670 | 67,201 | 255 | 121 | 255 | 122 | 1056 |
| GitTables (Numeric) | 72,636 | - | 219 | - | 346 | 897 | 897 |

* Annotated with two vocabularies (the last two columns: Schema.org ↑, DBpedia ↓)

*SOTAB* - The benchmark [39] provides CTA annotations for 45,834 tables and CPA annotations for 30,220 tables. The columns and column pairs were annotated using two vocabularies, Schema.org and DBpedia, resulting in two datasets but with shared tables. Henceforth, these datasets are abbreviated as SOTAB-SCH and SOTAB-DBP, respectively. Each column (pair) was annotated with one single label. We constructed two graphs for SOTAB-SCH and SOTAB-DBP, respectively, each serving as the label space for its dataset. The benchmark also provides splits from the full test set for the challenge topics.

*WikiTables-TURL* - The benchmark [14] is another large-scale dataset derived from WikiTable corpus [53], containing Wikipedia tables. It includes 406,706 tables for CTA annotation and 55,970 tables for CPA annotation. Freebase's types and properties [54] were used for annotation, with each column or column pair assigned at least one label, often multiple labels. We built a graph based on this vocabulary, leveraging the co-occurrence of type and property annotations.

*GitTables-Numeric* - GitTables [40] is a large-scale corpus of relational tables extracted from CSV files found in GitHub repositories. As GitTables contains rich numerical content that poses real-world challenges, we followed the extraction pipeline from Pythagoras [26] to identify numerical tables in GitTables, resulting in a corpus of 6,577 tables with an average of 2.08 textual and 8.96 numerical columns per table, covering a total of 219 semantic types.

### B. Baselines

Our architecture was compared with widely recognized baselines, later detailed in Section VII. Sato [7] and Sherlock [6] are among the earliest yet classic works in annotating columns; TURL [14] and DODUO [4] are well-established and representative works in this field since language models were adapted for column annotations; DODUO & TorchicTab [23], Watchog [55], and Pythagoras [26] are the current state-of-the-art models on the three respective datasets. Regarding the comparison between LLMs, instruction-based approaches such as GPT-3.5 and GPT-4 [56] serve as strong baselines, particularly when limited training tables are available. The baseline performance numbers in Section V-D are taken from the benchmark[1] and prior works [32, 4, 55, 26].

[1]https://webdatacommons.org/structureddata/sotab/v2/

TABLE III: Performance on SOTAB Dataset V2

| Method | Schema.org | | | | DBpedia | | | |
|---|---|---|---|---|---|---|---|---|
| | CTA | | CPA | | CTA | | CPA | |
| | Macro F1 | Micro F1 | Macro F1 | Micro F1 | Macro F1 | Micro F1 | Macro F1 | Micro F1 |
| Random Forest | - | 69.09 | - | 57.00 | - | 68.86 | - | 58.18 |
| TURL | 80.42 | 81.68 | 80.95 | 83.33 | 76.87 | 76.65 | 84.33 | 85.73 |
| DODUO | - | 86.60 | - | 90.21 | - | 87.93 | - | **93.39** |
| TorchicTab | - | 89.66 | - | 87.11 | - | 89.72 | - | 90.49 |
| RODEO w/o GNN - hardest ( $\sigma_{\text{task}} = 1$ ) | 56.75 | 62.18 | 51.00 | 57.22 | 80.00 | 80.76 | 77.85 | 80.20 |
| RODEO w/o GNN - semi-hard ( $\sigma_{\text{task}} = 0$ ) | 88.19 | 90.01 | 87.57 | 89.57 | 87.32 | 88.84 | 90.16 | 91.37 |
| RODEO w/o GNN - our method ( $0 < \sigma_{\text{task}} < 1$ ) | 88.15 | 90.38 | 89.74 | 91.07 | 90.67 | 91.04 | 90.72 | 92.15 |
| RODEO w/ GNN - hardest ( $\sigma_{\text{task}} = 1$ ) | 87.97 | 90.11 | 88.48 | 90.09 | 89.66 | 90.16 | 89.32 | 91.15 |
| RODEO w/ GNN - semi-hard ( $\sigma_{\text{task}} = 0$ ) | 88.53 | 90.28 | 89.16 | 90.64 | **91.51** | **91.78** | 90.06 | 91.59 |
| RODEO w/ GNN - our method ( $0 < \sigma_{\text{task}} < 1$ ) | **90.4** | **91.84** | **89.87** | **91.2** | 90.73 | 90.9 | **91.66** | 93.03 |

* Underline represents the runner up; the abbreviation "RODEO w/o GNN" denotes the variant of RODEO without the GNN, while "RODEO w/ GNN" denotes the full version.

### C. Experimental Settings

We used the AdamW optimizer [57] during training. Triplet loss with a soft margin [58] was applied with embedding size set to 64, where the ratio of negative mining strategies was set to 0.5 per epoch for CTA and 0.33 per epoch for CPA, as later detailed in Section VI-A. The model was trained for 40 epochs for each task, the learning rate was initially set to $1 \times 10^{-5}$ for 10 epochs and then decayed linearly. We use BERT-base with full parameter tuning with a subtable sampling strategy to address the input length limit [23]. To enable LLM training, we applied a LoRA adapter [59, 60] with a dropout rate of 0.4, a rank of 64 and an alpha value of 128.

Models are trained with table metadata on GitTables (Numeric), without metadata on SOTAB and WikiTables, and are all compared to baselines under the same conditions. For SOTAB and GitTables, we selected the label embedding closest to the column (or column pair) embeddings, as each column (or column pair) has only one single semantic annotation. The WikiTable dataset annotated each column or column pair with multiple semantic labels; in this case, we selected the top embeddings with the hierarchy for filtering.

### D. Main Results

(1) Table III presents the results on SOTAB Dataset V2, covering Schema.org and DBpedia divisions.

*Findings' summary* - (i) RODEO with the GNN variant significantly outperforms the state-of-the-art methods in both tasks, with improvements varying from 0.99% to 2.18% in Micro F1; (ii) our unified negative mining strategy consistently surpasses the semi-hard negative strategy under the same conditions, with improvements varying from 0.37% to 2.2% in Micro F1; (iii) adding the graph constraints consistently leads to better and more stable performance for both tasks compared to the variant without GNNs, with improvements varying from 0.13% to 2.94% in Micro F1.

*Low-Resource Scenarios* - To evaluate model performance with limited training data, we found an LLM benchmark [32] using the downsampled dataset from SOTAB [39], including roughly 1,200 training tables and 2,340 unseen column pairs for evaluation ( the same SOTAB-SCH test split for CPA). We evaluated RODEO against both zero-shot and fine-tuned *gpt-3.5-turbo-0613* and zero-shot *gpt-4-0125-preview*, as shown in Table IV. With such limited data, RODEO-BERT outperformed models of the same scale by a large margin, beating DODUO by about 10%, and it can serve as a low-cost alternative to GPT models, e.g., *gpt-4-0125-preview*, being at least 100 $\times$ smaller and requiring less computational resources and financial budget. Scaling up RODEO from BERT to large language models, RODEO-Qwen3, the model variant using Qwen3-8B embeddings, outperformed all other zero-shot and fine-tuned GPT models, demonstrating the scalability and robustness of our framework. This also showcases the promising usage of LLM embeddings for column annotation, not limited to instruction-based large language models.

TABLE IV: CPA Performance on downsampled SOTAB V2

| Method | Params | Setting | Micro F1 |
|---|---|---|---|
| TURL | 14.5M | fine-tuned | 60.75 |
| DODUO | 110M | fine-tuned | 70.34 |
| RoBERTA | 110M | fine-tuned | 71.45 |
| GPT-3.5 | ↑ (B) | zero-shot | 66.49 |
| GPT-3.5 | ↑ (B) | fine-tuned | 83.55 |
| GPT-4 | ↑↑ (B) | zero-shot | 81.43 |
| RODEO - BERT | ∼110M | fine-tuned | 81.54 |
| RODEO - Qwen3 | ∼0.6B | fine-tuned | 83.15 |
| RODEO - Qwen3 | ∼4B | fine-tuned | 83.76 |
| RODEO - Qwen3 | ∼8B | fine-tuned | **85.09** |

* Closed-source LLMs do not disclose parameters [56], estimated at billion scale.

*Challenging Splits* - To measure how well the model handles specific challenges, we evaluate RODEO using the specialized test splits from SOTAB as shown in Fig.5. For columns with missing values, RODEO reaches the state-of-the-art performance in CTA and with a small drop in CPA, indicating that sparse data poses a challenge where edges cannot be effectively matched without sufficient content; For columns that

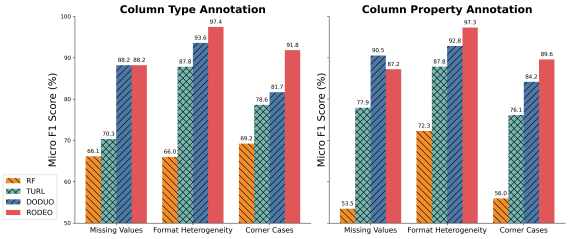



Fig. 5: Performance on SOTAB specific challenges

contain different date formats or numerical metrics, RODEO leads all the baselines and outperforms DODUO by around 4% in both tasks, as these types and properties are strongly related to certain table topics and the inter-table information can be leveraged through the graph; For corner cases with similar columns having different annotations and dissimilar columns sharing the same label, RODEO outperforms all the baselines and beats DODUO in CTA by about 10%. In the former case, it requires leveraging inter-table information, while in the latter, it requires intra-table information, RODEO can successfully handle both scenarios.

(2) Table V presents the results on WikiTables. To utilize all the annotations for multi-label training, we adopted a straightforward approach where multiple triples were created for all the positive labels given a column or column pair, as each triple can include only one positive label at a time.

TABLE V: Performance on WikiTables Dataset

| Method | CTA | | | CPA | | |
|---|---|---|---|---|---|---|
| | P | R | F1 | P | R | F1 |
| Sherlock | 88.40 | 70.55 | 78.47 | - | - | - |
| TURL | 90.54 | 87.23 | 88.86 | 91.18 | 90.69 | 90.94 |
| DODUO | 92.69 | **92.21** | 92.45 | 91.97 | 91.47 | 91.72 |
| Watchog | - | - | **92.53** | - | - | 92.38 |
| RODEO | **93.53** | 90.71 | 92.10 | **95.14** | **91.52** | **93.29** |

* P, R, and F1 denote Precision, Recall, and the Micro F1 score.

Regarding *column type annotation*, RODEO outperforms TURL and approaches the state-of-the-art performance with a Micro F1 score of 92.1%, comparable to models like DODUO and Watchog. We filtered out noise and resolved semantic conflicts in the inference results by leveraging the type hierarchy to ensure semantic consistency. For example, RODEO may infer an ordered list of types for target column ["*sports.pro_athlete*", "*soccer.football_player*", "*tennis.tennis_player*", "*people.person*"], where a higher position indicates a shorter distance to the target column. We prioritized the semantic type whose embedding is closer to the column embedding and used it along with the type hierarchy to determine which semantic types should be kept or discarded. In this case, we exclude "*tennis.tennis_player*" since the same hierarchy level "*soccer.football_player*" ranks higher. Regarding *column property annotation*, RODEO outperforms all the state-of-the-art methods with a Micro F1 score of 93.29%, achieving higher precision compared to other baselines. We also leverage the property hierarchy to filter the noise. Our results demonstrate that both the constraints between types and properties, and the hierarchies among types and among properties, contribute to the performance in the multi-label column annotation tasks.

(3) Table VI presents the results on GitTables, without property annotations but synthesized topics (Section IV-E).

TABLE VI: Performance on GitTables (numeric) Dataset

| Method | support weighted F1 | | | Macro-F1 | | |
|---|---|---|---|---|---|---|
| | numerical | textual | all | numerical | textual | all |
| Sherlock | 72.5 | 98.9 | 77.5 | 42.3 | 76.3 | 52.4 |
| Sato | 73.3 | 99.1 | 78.1 | 44.3 | 70.7 | 49.1 |
| DODUO | 76.1 | **99.2** | 80.4 | 40.9 | 74.9 | 48.9 |
| GPT-3.5 | 53.1 | 93.8 | 61.0 | 14.3 | 27.7 | 21.1 |
| Pythagoras | 81.3 | 99.0 | 84.6 | 47.6 | 89.3 | 54.4 |
| RODEO | **94.5** | **99.2** | **95.3** | **83.9** | **91.8** | **85.0** |

RODEO outperforms all state-of-the-art methods by more than 10% in support weighted F1 score and 30% in Macro-F1, showing outstanding capability in annotating numerical columns. In fact, 62.46% of types are linked to only one topic, and 72.43% of types are linked to two or fewer topics. This helps distinguish unique types by synthesized topics and implicitly narrows down candidates from the entire vocabulary to a small subset through the graph, where each cluster topic is connected to only a limited number of types. These structural constraints that other methods do not fully leverage help differentiate similar content across tables. This is especially important because many numerical columns appear similar.

## VI. Analysis

We perform in-depth ablation studies on triplet loss training and the graph neural network (GNN). We sample 1000 tables from the SOTAB-DBP, limiting each table to 100 rows, with a relatively balanced training dataset across categories.

### A. Cross Entropy Loss vs. Triplet Loss

To effectively evaluate the robustness of triplet loss in model training, we use the variant of RODEO without the GNN, defined in Section IV-D, to enable a fair comparison between triplet loss and cross-entropy loss, as used in DODUO [4].

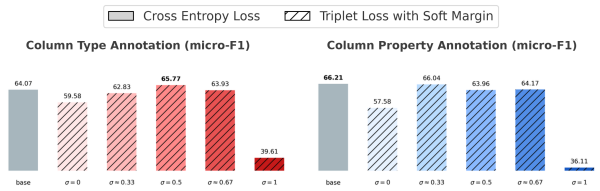



Fig. 6: Sensitivity analysis with ratio $\sigma$ (single-task)

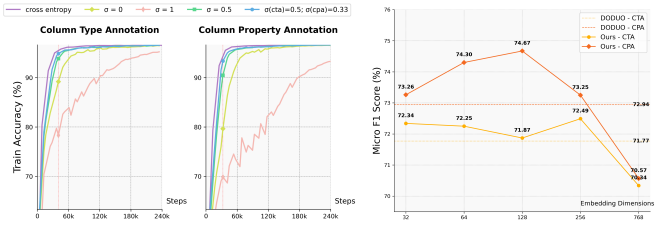
In the first experiment, shown in Fig.6, we apply single-task learning and compare results against DOSOLO, the single-task learning variant of DODUO [4]. This allows us to evaluate the impact of online negative mining on each task individually without mutual influence from multi-task learning. The ratio $\sigma$ determines the training strategy, indicating the percentage of batches optimized with semi-hard or hardest negatives per

epoch, with 0 using only semi-hard negatives and 1 using only hardest negatives. From Fig.6, we observe that (1) the semi-hard and hardest negative settings perform worse than our proposed method by mixing both conditions in Micro-F1; (2) the model's optimal ratio $\sigma$ varies by task - it performs best with an equal mix in CTA, while CPA benefits from a lower proportion of the hardest negatives.

TABLE VII: Ablation on negative mining (multi-task)

| Method | CTA | | CPA | |
|---|---|---|---|---|
| | Macro F1 | Micro F1 | Macro F1 | Micro F1 |
| DODUO | **67.15** | 71.77 | 66.10 | 72.94 |
| hardest ($\sigma_{\text{task}} = 1$) | 54.94 | 61.29 | 60.86 | 63.9 |
| semi-hard ($\sigma_{\text{task}} = 0$) | 65.04 | 69.73 | 62.19 | 68.55 |
| mixture ($\sigma_{\text{task}} = 0.5$) | 66.63 | 71.89 | 65.63 | 72.0 |
| mixture ($\sigma_{\text{cta}} = 0.5, \sigma_{\text{cpa}} = 0.33$) | 66.84 | **72.25** | **66.68** | **74.3** |

In the second experiment, we apply multi-task learning while keeping the model without GNN unchanged. In addition, we track the training accuracy to investigate how fast the model converges. The training accuracy curve represents the convergence speed, as reaching 100% training accuracy indicates that the model has correctly learned all the training examples. From the results in Table VII and Fig.7a, we observe that: (1) the previous findings from the first experiment still hold with the addition of multi-task learning; (2) customizing the ratio for each task can yield better results, surpassing the state-of-the-art baseline in Micro F1; (3) the convergence speed was largely improved in both tasks when applying our proposed negative mining strategy, it is closer to the convergence speed of using cross-entropy loss.




(a) Convergence speed (b) Embedding dimension

Fig. 7: Analysis of triplet loss training (multi-task)

The hardest negatives, often associated with local minima in previous works [48, 58, 61], arise likely due to the very large mining space. In human face identification, it is impractical to pair each human face with all dissimilar faces from dataset as negatives during the entire training, thereby missing opportunities for model to explore other unselected dissimilar faces. However, the mining space in column annotation is smaller and restricted to label vocabularies. This allows most types or properties to be easily selected as negatives and optimized during the early training stage, alleviating the local minima issue. Under such circumstances, we argue that the hardest negatives provide informative signals that semi-hard negatives fail to capture, contributing to the success of our method. Another ablation study is performed to investigate the impact of label embedding dimension. As shown in Fig. 7b, performance fluctuates between 32 and 256 dims but drops significantly at 768, suggesting that overly large embeddings add redundancy rather than useful information.

### *B. Model & Graph Variants (GNN vs. non-GNN)*

We conduct experiments to compare three architectures: RODEO, its ablation variant, and DODUO. Since neither the ablation variant nor DODUO utilize graph constraints, this setup enables us to assess the impact of graph constraints.

TABLE VIII: Ablation study on graph constraints

| Method | CTA | | CPA | |
|---|---|---|---|---|
| | Macro F1 | Micro F1 | Macro F1 | Micro F1 |
| DODUO | 67.15 | 71.77 | 66.10 | 72.94 |
| w/o GNN | 66.84 | 72.25 | 66.68 | 74.3 |
| w/ GNN ($\ell = 1$) | 71.32 | 73.44 | 66.88 | 73.34 |
| w/ GNN ($\ell = 2$) | 72.72 | 75.16 | 65.40 | 72.71 |
| w/ GNN ($\ell = 3$) | **73.55** | **76.04** | **69.79** | 75.08 |
| w/ GNN ($\ell = 4$) | 72.59 | 74.70 | 66.01 | 73.78 |
| w/o synthetic properties | 73.24 | 75.03 | 68.45 | **75.44** |
| w/ all synthetic properties | 71.74 | 74.38 | 67.52 | 73.66 |

To find the optimal number of GNN layers, we test it with varying layer counts. As shown in Table VIII, adding a single GNN layer immediately improves the performance in CTA. Adding more layers further enhances the performance, but it declines after three layers. Within SOTAB-DBP, a 3-hop neighborhood may be sufficient for identifying a type, as additional layers may introduce confusion or exceed the architecture's capacity. Nonetheless, the optimal number may vary across datasets, depending on the graph structure in the label space, following the principle that GNN complexity enhances performance up to a point before causing a decline.

We observe that the gain is more consistently significant in CTA than CPA as node embeddings act as the bridge in multi-task learning and column embeddings remain simple without fusion before matching, unlike column pair embeddings. We also attribute this to differences in label space expansion, as illustrated in Table II. The CTA label space expands from 46 types to 59 nodes, remaining relatively stable, while the label space in CPA grows substantially from 49 properties to 228 duplicated edges (with 64 unique edge types), approximately 4.5 times larger. This label expansion, with more candidates during inference, likely leads to additional challenges.

We also conducted a sensitivity analysis to examine synthetic properties and evaluate the model robustness under three graph variants, as these synthetic properties are derived solely from type co-occurrence, which does not ensure explicit CPA annotations. In this analysis, "w/ GNN ($\ell = 3$)" serves as the default graph, which includes only high-confidence triples with synthetic properties. The comparison results indicate RODEO is reasonably robust to imperfect label graphs and can benefit from removing low-confidence triples. As not all triples with synthetic properties contribute positively, some may introduce noise and degrade performance, highlighting the importance of graph quality in achieving better performance.

### C. Time and Memory Costs

TABLE IX: Time and memory cost analysis

| Cost | DODUO | RODEO |
|---|---|---|
| Time (training) | $\mathcal{O}((Nd_h^2 + N^2 d_h) \cdot L_1 + d_h \cdot \lvert S\rvert)$ | $\mathcal{O}((Nd_h^2 + N^2 d_h) \cdot L_1 + (\hat{n} + \hat{m})d^2 \cdot L_2 + d \cdot (d_h + \lvert\hat{S}\rvert))$ |
| Time (inference) | $\mathcal{O}((Nd_h^2 + N^2 d_h) \cdot L_1 + d_h \cdot \lvert S\rvert)$ | $\approx \mathcal{O}((Nd_h^2 + N^2 d_h) \cdot L_1 + d \cdot (d_h + \lvert\hat{S}\rvert))$ |
| Memory (training) | $\mathcal{O}((Nd_h + N^2 h) \cdot L_1 + d_h \cdot \lvert S\rvert)$ | $\mathcal{O}((Nd_h + N^2 h) \cdot L_1 + (\hat{n} + \hat{m})d \cdot L_2 + d \cdot (d_h + \hat{m} + \hat{n}))$ |
| Memory (inference) | $\mathcal{O}((Nd_h + N^2 h) \cdot L_1 + d_h \cdot \lvert S\rvert)$ | $\approx \mathcal{O}((Nd_h + N^2 h) \cdot L_1 + d \cdot (d_h + \lvert\hat{S}\rvert))$ |

Regarding time complexity, given a table serialized into $N$ tokens and fed into DODUO (a transformer, $L_1$ layers, $h$ heads, $d_h$ dim) for classification ($m$ types, $n$ properties), it costs $\mathcal{O}((Nd_h^2 + N^2 d_h) \cdot L_1 + d_h \cdot |S|)$, where $|S| \in \{m, n\}$, for one forward pass. In RODEO, the GNN ($L_2$ layers, $\hat{m}$ nodes, $\hat{n}$ edges, $d$ dim) and embedding matching add overhead $\mathcal{O}((\hat{m}+\hat{n})d^2 \cdot L_2)$ + $\mathcal{O}(d_h \cdot d + d \cdot |\hat{S}|)$ where $|\hat{S}| \in \{\hat{m}, \hat{n}\}$ and $d_h \cdot d$ denotes the projection layer that resizes the embeddings. In addition, negative mining costs $\mathcal{O}(|\hat{S}|)$ during training.

Regarding space complexity, the transformer memory cost grows quadratically with tokens, $\mathcal{O}\big((Nd_h + N^2 h) \cdot L_1\big)$, while the cost scales linearly with nodes and edges in GNN, $\mathcal{O}((\hat{m} + \hat{n})d \cdot L_2)$, with $\hat{m} + \hat{n}$ remaining small in column annotation. The classification requires $\mathcal{O}(d_h \cdot |S|)$ whereas embedding matching requires $\mathcal{O}(d_h \cdot d + d \cdot (\hat{m}+\hat{n}))$ in memory.

However, the GNN-related complexity mainly affects the training phase. At inference time, the GNN can be run once independently with the resulting label embeddings cached to avoid repeated GNN computation since the graph is fixed, also no negative mining is required, making the inference cost comparable to DODUO for both time and memory (with only marginal overhead from table-to-graph embedding matching).

## VII. Related Work

Column type annotation models leverage advances in *neural networks*. Sherlock [6], a multi-input deep neural network, outperformed traditional methods by combining statistical features and textual embeddings. Sato [7] considered not only intra-table patterns from an ensemble of embeddings but also context from latent topic model [62] for type annotations.

With the trend of the pre-training, *BERT-like language models* [8, 9, 10] replaced small neural networks in modeling tables and were widely evaluated in various tabular challenges [12, 13, 14, 15, 4, 16, 55, 63]. One representative, TURL [14], utilizing row-based serialization, achieved impressive results in type and property annotation by learning deep contextualized representations on relational tables via pre-training. While DODUO [4] serialized cell values within the same columns as sentences and generated column embeddings, the multi-task learning paradigm implicitly transferred knowledge between type and property annotations. TorchicTab [23] proposed a subtable sampling strategy to address the limitation of input length during training. RECA [22] also mitigated this limitation and retrieved related tables to improve column annotation quality. Watchog [55] learned robust column representations by leveraging a large-scale unlabeled table corpus with contrastive learning. To model tables in RODEO, a *BERT-like language model* is first integrated, as widely adopted in previous works, but trained along with graph neural networks using triplet loss for matching rather than cross-entropy.

Utilizing *GPT-like large language models* also achieved strong performance in table annotation [31, 32, 33, 34, 35, 36, 37], where tables are provided in a prompt and annotations are generated as text. Since RODEO is an embedding-based training framework agnostic to the table encoder, large language models (LLMs) are leveraged to generate embeddings rather than text, a usage of LLMs that has not been fully explored in previous research for column annotation. Our work is in a fully-supervised setting, also different from previous works that soley focus on zero-shot or few-shot learning [64].

The aforementioned works focused on information solely from table side, past works also explored *integrating external knowledge bases (KB)*. ColNet [17] applied a convolutional neural network for type prediction on word2vec embeddings [65] of entities obtained from KB reasoning and lookup; HNN [18] combined tabular features from a hybrid neural network with vectors extracted by KB lookup to predict types. Scaling up to language models, KGLink [24] also combined WikiData information with language models to alleviate the problems casued by type granularity and missing table context. These KB-enhanced frameworks all successfully incorporated entity-level knowledge. This knowledge enhances intra-table understanding about what the columns in this table mean, but provides less about how the type and property annotations relate across different tables. Thus, we seek a graph that provides a global view of all types and properties. Unlike creating graphs with individual tables like SANTOS [51] and Pythagoras [26], in this paper, we construct one graph per dataset that spans all its tables, capturing both intra-table and inter-table context. RODEO incorporates label constraint across and within tables via GNNs, while preserving table understanding capabilities of the language model. Since the graph is pre-constructed using a pre-defined vocabulary, for incremental updates, handling newly introduced labels would require a dynamic graph setting [66]; in order to keep RODEO focused, we restrict our study to the static graph in this paper.

## VIII. Conclusion

In this paper, we propose RODEO that leverages label constraints via the graph for column annotations. Experiments on three datasets show that RODEO reaches new state-of-the-art performance. Our analyses further confirm that adding graph constraints consistently leads to better performance, while the training framework remains robust and efficient. We also scale up RODEO with Qwen Embeddings, demonstrating the scalability of our embedding-based approach and its promising potential for future use in the era of LLMs. As future work, we also plan to investigate incorporating entity-level information from rich external knowledge to further enhance RODEO. We believe our research opens up a new perspective for understanding tables through the graph, provides a strong machine learning foundation for future studies, and can effectively address diverse real-world data management challenges.

## AI-Generated Content Acknowledgement

In this paper, we use ChatGPT to assist in visualizing partial results, specifically for generating matplotlib code for embedding visualization, also other line and bar charts.

## References


[1] S. Zhang and K. Balog, "Web table extraction, retrieval and augmentation," in *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '19. ACM, Jul. 2019. [Online]. Available: http://dx.doi.org/10.1145/3331184.3331385

[2] J. Pujara, P. Szekely, H. Sun, and M. Chen, "From tables to knowledge: Recent advances in table understanding," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery; Data Mining*, ser. KDD '21. ACM, Aug. 2021, p. 4060–4061. [Online]. Available: http://dx.doi.org/10.1145/3447548.3470809

[3] J. Liu, Y. Chabot, R. Troncy, V.-P. Huynh, T. Labbé, and P. Monnin, "From tabular data to knowledge graphs: A survey of semantic table interpretation tasks and methods," *Journal of Web Semantics*, vol. 76, p. 100761, Apr. 2023.

[4] Y. Suhara, J. Li, Y. Li, D. Zhang, Ç. Demiralp, C. Chen, and W. Tan, "Annotating columns with pre-trained language models," in *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*. ACM, 2022, pp. 1493–1503. [Online]. Available: https://doi.org/10.1145/3514221.3517906

[5] G. Badaro, M. Saeed, and P. Papotti, "Transformers for tabular data representation: A survey of models and applications," *Transactions of the Association for Computational Linguistics*, vol. 11, p. 227–249, 2023. [Online]. Available: http://dx.doi.org/10.1162/tacl_a_00544

[6] M. Hulsebos, K. Z. Hu, M. A. Bakker, E. Zgraggen, A. Satyanarayan, T. Kraska, Ç. Demiralp, and C. A. Hidalgo, "Sherlock: A deep learning approach to semantic data type detection," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*. ACM, 2019, pp. 1500–1508. [Online]. Available: https://doi.org/10.1145/3292500.3330993

[7] D. Zhang, Y. Suhara, J. Li, M. Hulsebos, Ç. Demiralp, and W. Tan, "Sato: Contextual semantic type detection in tables," *Proc. VLDB Endow.*, vol. 13, no. 11, pp. 1835–1848, 2020. [Online]. Available: http://www.vldb.org/pvldb/vol13/p1835-zhang.pdf

[8] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 2019, pp. 4171–4186. [Online]. Available: https://doi.org/10.18653/v1/n19-1423

[9] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized BERT pretraining approach," *CoRR*, vol. abs/1907.11692, 2019. [Online]. Available: http://arxiv.org/abs/1907.11692

[10] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A lite BERT for self-supervised learning of language representations," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. [Online]. Available: https://openreview.net/forum?id=H1eA7AEtvS

[11] M. Hulsebos, X. Deng, H. Sun, and P. Papotti, "Models and practice of neural table representations," in *Companion of the 2023 International Conference on Management of Data*, ser. SIGMOD/PODS '23. ACM, jun 2023, p. 83–89. [Online]. Available: http://dx.doi.org/10.1145/3555041.3589411

[12] P. Yin, G. Neubig, W. Yih, and S. Riedel, "Tabert: Pretraining for joint understanding of textual and tabular data," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*. Association for Computational Linguistics, 2020, pp. 8413–8426. [Online]. Available: https://doi.org/10.18653/v1/2020.acl-main.745

[13] Z. Wang, H. Dong, R. Jia, J. Li, Z. Fu, S. Han, and D. Zhang, "TUTA: tree-based transformers for generally structured table pre-training," in *KDD '21: The 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, Singapore, August 14-18, 2021*. ACM, 2021, pp. 1780–1790. [Online]. Available: https://doi.org/10.1145/3447548.3467434

[14] X. Deng, H. Sun, A. Lees, Y. Wu, and C. Yu, "TURL: table understanding through representation learning," *Proc. VLDB Endow.*, vol. 14, no. 3, pp. 307–319, 2020. [Online]. Available: http://www.vldb.org/pvldb/vol14/p307-deng.pdf

[15] H. Iida, D. Thai, V. Manjunatha, and M. Iyyer, "TABBIE: pretrained representations of tabular data," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*. Association for Computational Linguistics, 2021, pp. 3446–3456. [Online]. Available: https://doi.org/10.18653/v1/2021.naacl-main.270

[16] J. Tu, J. Fan, N. Tang, P. Wang, G. Li, X. Du, X. Jia, and S. Gao, "Unicorn: A unified multi-tasking model for supporting matching tasks in data integration," *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, p. 1–26, May 2023. [Online]. Available: http://dx.doi.org/10.1145/3588938

[17] J. Chen, E. Jiménez-Ruiz, I. Horrocks, and C. Sutton, "Colnet: Embedding the semantics of web tables for column type prediction," in *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27 - February 1, 2019*. AAAI Press, 2019, pp. 29–36. [Online]. Available: https://doi.org/10.1609/aaai.v33i01.330129

[18] J. Chen, E. Jimenez-Ruiz, I. Horrocks, and C. Sutton, "Learning semantic annotations for tabular data," in *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, S. Kraus, Ed. ijcai.org, 2019, pp. 2088–2094. [Online]. Available: https://doi.org/10.24963/ijcai.2019/289

[19] K. Takeoka, M. Oyamada, S. Nakadai, and T. Okadome, "Meimei: An efficient probabilistic approach for semantically annotating tables," in *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27 - February 1, 2019*. AAAI Press, 2019, pp. 281–288. [Online]. Available: https://doi.org/10.1609/aaai.v33i01.3301281

[20] E. Rahm and P. A. Bernstein, "A survey of approaches to automatic schema matching," *The VLDB Journal*, vol. 10, no. 4, p. 334–350, Dec. 2001. [Online]. Available: http://dx.doi.org/10.1007/s007780100057

[21] Z. Chen, M. Trabelsi, J. Heflin, Y. Xu, and B. D. Davison, "Table search using a deep contextualized language model," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '20. ACM, Jul. 2020, p. 589–598. [Online]. Available: http://dx.doi.org/10.1145/3397271.3401044

[22] Y. Sun, H. Xin, and L. Chen, "RECA: Related tables enhanced column semantic type annotation framework," *Proc. VLDB Endow.*, vol. 16, no. 6, pp. 1319–1331, 2023. [Online]. Available: https://www.vldb.org/pvldb/vol16/p1319-sun.pdf

[23] I. Dasoulas, D. Yang, X. Duan, and A. Dimou, "Torchictab: Semantic table annotation with wikidata and language models," in *Proceedings of the Semantic Web Challenge on Tabular Data to Knowledge Graph Matching, SemTab 2023, co-located with the 22nd International Semantic Web Conference, ISWC 2023, Athens, Greece, November 6-10, 2023*, ser. CEUR Workshop Proceedings, vol. 3557. CEUR-WS.org, 2023, pp. 21–37. [Online]. Available: https://ceur-ws.org/Vol-3557/paper2.pdf

[24] Y. Wang, H. Xin, and L. Chen, "Kglink: A column type annotation method that combines knowledge graph and pre-trained language model," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, May 2024, p. 1023–1035. [Online]. Available: http://dx.doi.org/10.1109/icde60146.2024.00083

[25] Y. Zhao, Y. Li, C. Li, and R. Zhang, "Multihiertt: Numerical reasoning over multi hierarchical tabular and textual data," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2022. [Online]. Available: http://dx.doi.org/10.18653/v1/2022.acl-long.454

[26] S. Langenecker, C. Sturm, C. Schalles, and C. Binnig, "Pythagoras: Semantic type detection of numerical data in enterprise data lakes." in *Proceedings of the 27th International Conference on Extending Database Technology (EDBT)*, 2024, pp. 725–733. [Online]. Available: http://dx.doi.org/10.48786/edbt.2024.62

[27] W. Fan, L. Geng, R. Jin, P. Lu, R. Tugay, and W. Yu, "Linking entities across relations and graphs," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, May 2022, p. 634–647. [Online]. Available: http://dx.doi.org/10.1109/icde53745.2022.00052

[28] B. Genossar, R. Shraga, and A. Gal, "Flexer: Flexible entity resolution for multiple intents," *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, p. 1–27, May 2023. [Online]. Available: http://dx.doi.org/10.1145/3588722

[29] R. Cappuzzo, S. Thirumuruganathan, and P. Papotti, "Relational data imputation with graph neural networks," in *EDBT/ICDT 2024, 27th International Conference on Extending Database Technology, 25-28 March 2024, Paestum, Italy*, Paestum, 2024. [Online]. Available: https://openproceedings.org/2024/conf/edbt/paper-62.pdf

[30] C. Koutras, J. Zhang, X. Qin, C. Lei, V. Ioannidis, C. Faloutsos, G. Karypis, and A. Katsifodimos, "Omnimatch: Joinability discovery in data products," *Proceedings of the VLDB Endowment*, vol. 18, no. 11, p. 4588–4601, Jul. 2025. [Online]. Available: http://dx.doi.org/10.14778/3749646.3749715

[31] K. Korini and C. Bizer, "Column type annotation using chatgpt," in *Joint Proceedings of Workshops at the 49th International Conference on Very Large Data Bases (VLDB 2023), Vancouver, Canada, August 28 - September 1, 2023*, ser. CEUR Workshop Proceedings, vol. 3462. CEUR-WS.org, 2023. [Online]. Available: https://ceur-ws.org/Vol-3462/TADA1.pdf

[32] K. Korini and C. Bize-r, "Column property annotation using large language models," in *The Semantic Web: ESWC 2024 Satellite Events*. Springer Nature Switzerland, 2025, p. 61–70. [Online]. Available: http://dx.doi.org/10.1007/978-3-031-78952-6_6

[33] B. Feuer, Y. Liu, C. Hegde, and J. Freire, "Archetype: A novel framework for open-source column type annotation using large language models," *Proceedings of the VLDB*

*Endowment*, vol. 17, no. 9, p. 2279–2292, May 2024. [Online]. Available: http://dx.doi.org/10.14778/3665844.3665857

[34] S. Hegselmann, A. Buendia, H. Lang, M. Agrawal, X. Jiang, and D. Sontag, "Tabllm: Few-shot classification of tabular data with large language models," in *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 206. PMLR, 25–27 Apr 2023, pp. 5549–5581. [Online]. Available: https://proceedings.mlr.press/v206/hegselmann23a.html

[35] T. Zhang, X. Yue, Y. Li, and H. Sun, "TableLlama: Towards open large generalist models for tables," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, Jun. 2024, pp. 6024–6044. [Online]. Available: https://aclanthology.org/2024.naacl-long.335

[36] P. Li, Y. He, D. Yashar, W. Cui, S. Ge, H. Zhang, D. Rifinski Fainman, D. Zhang, and S. Chaudhuri, "Table-gpt: Table fine-tuned gpt for diverse table tasks," *Proceedings of the ACM on Management of Data*, vol. 2, no. 3, p. 1–28, May 2024. [Online]. Available: http://dx.doi.org/10.1145/3654979

[37] K. Korini and C. Bizer, "Evaluating knowledge generation and self-refinement strategies for llm-based column type annotation," 2025. [Online]. Available: https://arxiv.org/abs/2503.02718

[38] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou, "Qwen3 embedding: Advancing text embedding and reranking through foundation models," *arXiv preprint arXiv:2506.05176*, 2025.

[39] K. Korini, R. Peeters, and C. Bizer, "SOTAB: the WDC schema.org table annotation benchmark," in *Proceedings of the Semantic Web Challenge on Tabular Data to Knowledge Graph Matching, SemTab 2022, co-located with the 21st International Semantic Web Conference, ISWC 2022, Virtual conference, October 23-27, 2022*, ser. CEUR Workshop Proceedings, vol. 3320. CEUR-WS.org, 2022, pp. 14–19. [Online]. Available: https://ceur-ws.org/Vol-3320/paper1.pdf

[40] M. Hulsebos, Ç. Demiralp, and P. Groth, "Gittables: A large-scale corpus of relational tables," *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, p. 1–17, May 2023. [Online]. Available: http://dx.doi.org/10.1145/3588710

[41] X. Wang, T. Gao, Z. Zhu, Z. Zhang, Z. Liu, J. Li, and J. Tang, "Kepler: A unified model for knowledge embedding and pre-trained language representation," *Transactions of the Association for Computational Linguistics*, vol. 9, p. 176–194, Mar. 2021. [Online]. Available: http://dx.doi.org/10.1162/tacl_a_00360

[42] X. Bresson and T. Laurent, "Residual gated graph convnets," *ArXiv*, vol. abs/1711.07553, 2017. [Online]. Available: https://arxiv.org/abs/1711.07553

[43] V. P. Dwivedi, C. K. Joshi, A. T. Luu, T. Laurent, Y. Bengio, and X. Bresson, "Benchmarking graph neural networks," *J. Mach. Learn. Res.*, vol. 24, pp. 43:1–43:48, 2023. [Online]. Available: http://jmlr.org/papers/v24/22-0567.html

[44] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://openreview.net/forum?id=SJU4ayYgl

[45] D. Marcheggiani and I. Titov, "Encoding sentences with graph convolutional networks for semantic role labeling," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2017. [Online]. Available: http://dx.doi.org/10.18653/v1/d17-1159

[46] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Jun. 2016. [Online]. Available: http://dx.doi.org/10.1109/cvpr.2016.90

[47] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," 2015.

[48] F. Schroff, D. Kalenichenko, and J. Philbin, "Facenet: A unified embedding for face recognition and clustering," in *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Jun. 2015. [Online]. Available: http://dx.doi.org/10.1109/cvpr.2015.7298682

[49] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, Y. W. Teh and M. Titterington, Eds., vol. 9. Chia Laguna Resort, Sardinia, Italy: PMLR, 13–15 May 2010, pp. 249–256. [Online]. Available: https://proceedings.mlr.press/v9/glorot10a.html

[50] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," in *Advances in Neural Information Processing Systems 32*, Vancouver, Canada, 2019. [Online]. Available: https://openreview.net/references/pdf?id=S1qBAf6rr

[51] A. Khatiwada, G. Fan, R. Shraga, Z. Chen, W. Gatterbauer, R. J. Miller, and M. Riedewald, "Santos: Relationship-based semantic table union search," *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, p. 1–25, May 2023. [Online]. Available: http://dx.doi.org/10.1145/3588689

[52] Y. Wang, D. He, F. Li, X. Long, Z. Zhou, J. Ma, and S. Wen, "Multi-label classification with label graph superimposing," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 07, pp. 12 265–12 272, Apr. 2020. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/6909

[53] C. S. Bhagavatula, T. Noraset, and D. Downey, *TabEL:*

*Entity Linking in Web Tables*. Springer International Publishing, 2015, p. 425–441. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-25007-6_25

[54] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor, "Freebase: a collaboratively created graph database for structuring human knowledge," in *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '08. New York, NY, USA: Association for Computing Machinery, 2008, p. 1247–1250. [Online]. Available: https://doi.org/10.1145/1376616.1376746

[55] Z. Miao and J. Wang, "Watchog: A light-weight contrastive learning based framework for column annotation," *Proc. ACM Manag. Data*, vol. 1, no. 4, pp. 272:1–272:24, 2023. [Online]. Available: https://doi.org/10.1145/3626766

[56] OpenAI, "GPT-4 Technical Report," *arXiv e-prints*, Mar. 2023.

[57] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. [Online]. Available: https://openreview.net/forum?id=Bkg6RiCqY7

[58] Y. Zhai, X. Guo, Y. Lu, and H. Li, "In defense of the classification loss for person re-identification," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019, pp. 1526–1535.

[59] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=nZeVKeeFYf9

[60] Y. Lin, X. Ma, X. Chu, Y. Jin, Z. Yang, and Y. Wang, "Lora dropout as a sparsity regularizer for overfitting reduction," *Knowledge-Based Systems*, p. 114241, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950705125012821

[61] Y. Yuan, W. Chen, Y. Yang, and Z. Wang, "In defense of the triplet loss again: Learning robust person re-identification with fast approximated triplet loss and label distillation," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. IEEE, Jun. 2020. [Online]. Available: http://dx.doi.org/10.1109/cvprw50498.2020.00185

[62] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent dirichlet allocation," *J. Mach. Learn. Res.*, vol. 3, pp. 993–1022, 2003. [Online]. Available: http://jmlr.org/papers/v3/blei03a.html

[63] J. Qiu, A. Song, J. Jin, J. Chen, X. Zhang, X. Fang, and T. Zhang, "Matching tabular data to knowledge graph with effective core column set discovery," *ACM Transactions on the Web*, vol. 18, no. 4, p. 1–27, Oct. 2024.

[64] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf

[65] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings*, 2013. [Online]. Available: http://arxiv.org/abs/1301.3781

[66] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. Schardl, and C. Leiserson, "Evolvegcn: Evolving graph convolutional networks for dynamic graphs," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, p. 5363–5370, Apr. 2020. [Online]. Available: http://dx.doi.org/10.1609/AAAI.V34I04.5984